

CipherShare System Documentation Report

Abstract

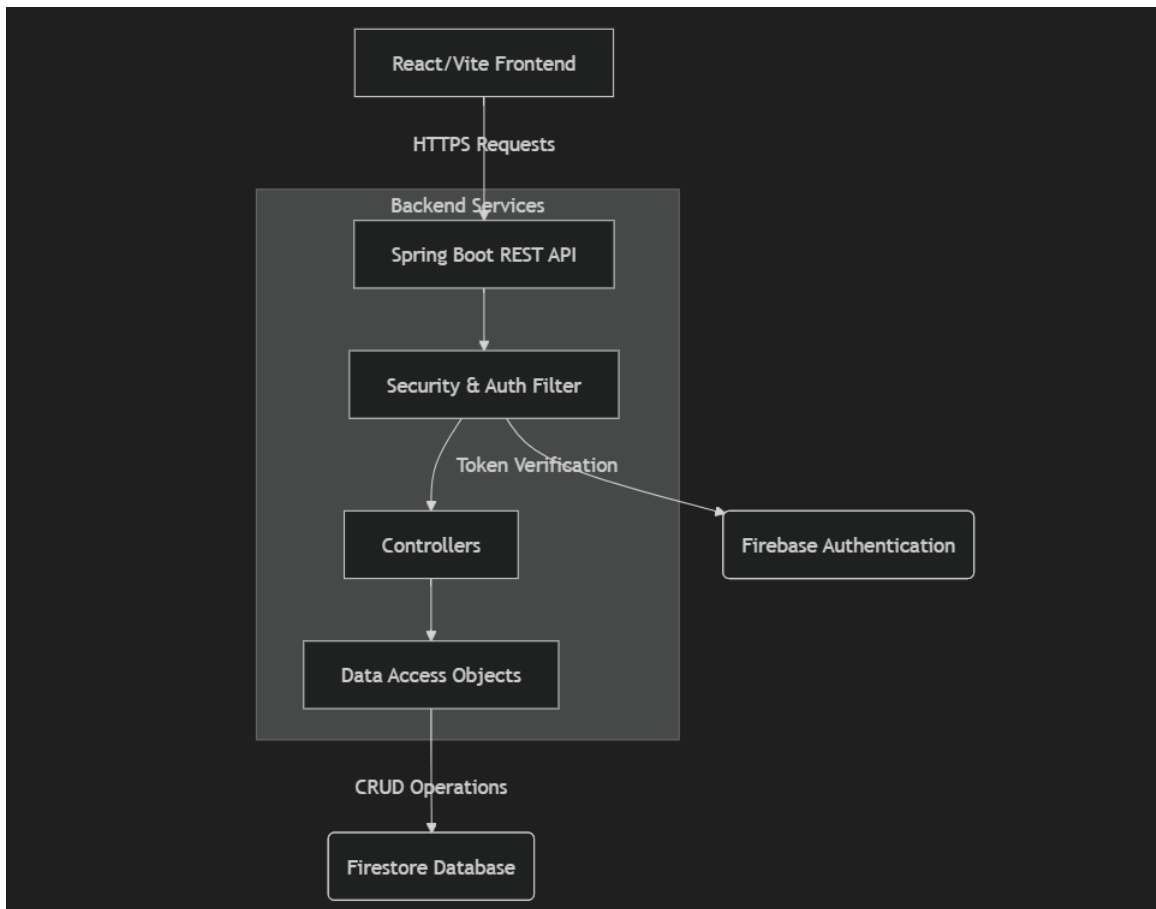
CipherShare is an enterprise-grade file-sharing system designed to ensure secure, role-based access to sensitive documents. By combining a React frontend, a Spring Boot REST API, and Firebase's authentication and database services, this platform provides a seamless and scalable solution for organizations. The system features multi-tier access control (Admin, Manager, and Employee) to govern file operations, mitigating the risk of unauthorized data exposure. This report outlines the architecture, technology stack, access control matrix, and future security enhancements of the CipherShare system.

1. Executive Summary

CipherShare is a secure, role-based enterprise file-sharing platform. It allows organizations to centrally manage files, ensuring that documents are only accessible to authorized personnel through strict Role-Based Access Control (RBAC). The system features an Admin command center, Manager team-file capabilities, and a secure User environment.

2. System Architecture

The application uses a decoupled client-server architecture, relying on React for the frontend, Spring Boot for the backend, and Firebase for authentication and database management.



3. Technology Stack

Frontend Architecture

- **Framework:** React 18 built with Vite for optimal performance.
- **Routing:** React Router v6 with custom <ProtectedRoute> components for role enforcement.
- **Styling:** Custom Vanilla CSS utilizing CSS variables for theme consistency and glassmorphism elements.
- **State Management:** React Context API (AuthContext) for global user and session state.

Backend Architecture

- **Framework:** Java 17 + Spring Boot 3
- **Server:** Embedded Apache Tomcat
- **Build Tool:** Maven (via Maven Wrapper mvnw)
- **Security:** Custom CORS configurations and Firebase Token verification filters.

Cloud Services (Firebase)

- **Authentication:** Handles user registration, login, and secure JWT generation.
- **Firestore:** NoSQL document database used to store User Profiles, File Metadata, and Activity Logs.

4. Role-Based Access Control (RBAC) Matrix

Security is enforced both on the client side (UI rendering and routing) and the server side (API endpoints).

Feature / Action	Admin 👑	Manager 🛠️	User/Employee 👤
Login & Dashboard Access	✓	✓	✓
View Shared Files	✓	✓	✓
Upload New Files	✓	✓	✗
Share Files with Others	✓	✓	✗
Manage Users (Approve/Reject)	✓	✗	✗
Update User Roles	✓	✗	✗
View Global Activity Logs	✓	✗	✗

NOTE

When a new user registers as a Manager, their account defaults to isActive = false and requires explicit approval from an Admin before they can log in.

5. Core Data Models

User

json

```
{
  "uid": "firebase_auth_uid",
  "name": "John Doe",
  "email": "john@example.com",
  "role": "manager",
  "isActive": true,
  "createdAt": "timestamp"
}
```

FileMetadata

json

```
{
  "fileId": "unique_uuid",
  "fileName": "Report.pdf",
  "fileSize": 1048576,
  "fileType": "application/pdf",
  "uploadedBy": "user_uid",
  "uploadDate": "timestamp",
  "sharedWith": ["user_uid_1", "user_uid_2"],
  "encryptionKey": "..."
}
```

6. Key Workflows

User Registration & Approval

1. User submits registration details via the frontend.
2. Firebase Auth creates the credential; a user document is created in Firestore.
3. If the requested role is manager, isActive is set to false.
4. Admins see the pending request in their dashboard and can approve or reject it.

Secure File Access

1. User navigates to their dashboard to see files shared with them.
2. Clicking a file sends a request to the backend with their Firebase JWT.
3. The Spring Boot backend validates the token, extracts the UID, and verifies that the UID exists in the file's sharedWith array.
4. If authorized, the file blob/metadata is returned to the client.

IMPORTANT

The backend explicitly denies access to files if the requesting user is not the owner, is not an admin, and is not in the sharedWith list.

7. Security Guidelines

- **Token Validation:** Every incoming request to the API must include a Bearer token in the Authorization header. The custom SecurityFilter intercepts requests and validates the token using the Firebase Admin SDK.
- **CORS Configuration:** The backend explicitly allows Cross-Origin Resource Sharing only from specific trusted frontend origins (e.g., <http://localhost:5173>) to prevent CSRF and unauthorized cross-domain requests.
- **Data Encryption:** All files uploaded through CipherShare should be stored securely. Metadata and sharing rules are isolated in Firestore collections with strict security rules enforcing authentication before read/write operations.

8. API Endpoints Reference

Below is a high-level overview of the backend REST controllers:

- **Admin Controller (/api/admin/*)**
 - GET /dashboard - Retrieves global statistics and recent logs.
 - POST /update-role - Modifies a user's role (Admin only).
 - POST /approve-user - Approves a pending Manager account.
- **File Controller (/api/files/*)**
 - POST /upload - Uploads a new document and records metadata.
 - POST /share - Updates the sharedWith array to grant access to another user.
 - GET /download/{fileId} - Initiates a secure file download stream.
- **User Controller (/api/user/*)**
 - GET /my-files - Retrieves files owned by or shared with the authenticated user.
 - GET /profile - Fetches the authenticated user's profile details.

9. Future Enhancements

- **Two-Factor Authentication (2FA):** Implementing multi-factor auth for Manager and Admin roles.
- **End-to-End Encryption (E2EE):** Encrypting file payloads on the client side before upload so that the server only handles ciphertexts.

- **Advanced Audit Filtering:** Adding granular search and export features for the Admin Activity Logs page.

10. Conclusion

CipherShare provides a robust, scalable, and highly secure environment for enterprise file management. By leveraging the speed of React, the reliability of Spring Boot, and the seamless authentication of Firebase, the platform ensures that sensitive data is strictly governed by its role-based access control models. The decoupled architecture also leaves the system well-positioned for future feature expansion and enterprise integrations.